

Fabricio Roskowski
Klaus Dieter Kupper

Avaliação 3 – Projeto RTL do Algoritmo de Máximo Divisor Comum

Itajaí – SC
Novembro de 2025

Fabricao Roskowski
Klaus Dieter Kupper

Avaliação 3 – Projeto RTL do Algoritmo de Máximo Divisor Comum

Relatório técnico apresentado como requisito parcial para a Avaliação 3 da disciplina Projeto de Sistemas Digitais, abordando o desenvolvimento RTL de um sistema digital para cálculo do Máximo Divisor Comum (MDC).

Universidade do Vale do Itajaí
Programa de Pós-Graduação em Computação Aplicada
Disciplina: Projeto de Sistemas Digitais

Orientador: Prof. Dr. Cesar Albenes Zeferino
Prof. Dr. Douglas Rossi Melo

Itajaí – SC
Novembro de 2025

Resumo

Este relatório descreve o desenvolvimento de um sistema digital baseado em VHDL para cálculo do Máximo Divisor Comum (MDC) entre dois operandos de 8 bits, seguindo o método de projeto RTL discutido em sala de aula. Partindo do algoritmo em linguagem C, foi derivada uma máquina de estados de alto nível, identificadas as partições de controle e caminho de dados, definidos os sinais internos com a convenção de nomenclatura solicitada e implementados os blocos VHDL correspondentes (topo, controle, caminho de dados e testbench). São apresentados os diagramas conceituais, a descrição do fluxo de dados, o planejamento de verificação com ModelSim e as orientações para síntese no Quartus Prime. Resultados de simulação e métricas de síntese serão incorporados após a execução dos experimentos.

Palavras-chave: VHDL, RTL, máquina de estados, MDC.

Sumário

1	INTRODUÇÃO	4
2	ESPECIFICAÇÃO DO SISTEMA	5
2.1	Resumo do Enunciado	5
2.2	Interface e Convenção de Sinais	5
3	METODOLOGIA DE PROJETO RTL	7
3.1	Algoritmo de Referência em C	7
3.2	Derivação da Máquina de Estados	8
3.3	Particionamento do Caminho de Dados	9
4	ARQUITETURA RTL PROPOSTA	10
4.1	Visão Geral	10
4.2	Bloco de Controle	10
4.3	Caminho de Dados	10
5	IMPLEMENTAÇÃO EM VHDL	11
5.1	Módulo Topo	11
5.2	Máquina de Controle	12
5.3	Caminho de Dados	15
5.4	Testbench	17
6	PLANO DE VALIDAÇÃO E SÍNTESE	19
6.1	Validação Funcional	19
6.2	Síntese e Análise RTL	19
7	CONCLUSÕES PARCIAIS	21
	Referências	21
	REFERÊNCIAS	22

1 Introdução

A Avaliação 3 da disciplina Projeto de Sistemas Digitais tem como objetivo consolidar o método de projeto RTL por meio da implementação de um algoritmo não trivial na linguagem VHDL. O algoritmo selecionado calcula o Máximo Divisor Comum (MDC) de dois operandos inteiros sem sinal de 8 bits utilizando o método de subtrações sucessivas. A especificação exige a decomposição em controle (máquina de estados) e caminho de dados, a adoção de uma convenção de nomenclatura padronizada, a verificação funcional por meio de testbenches no ModelSim e a síntese do sistema no Quartus Prime para obtenção de métricas de custo e desempenho.

Este documento registra o processo completo de desenvolvimento: interpretação dos requisitos, construção da máquina de estados de alto nível a partir do algoritmo em C, identificação dos blocos de controle e processamento, definição dos sinais internos, implementação em VHDL no estilo recomendado pela Xilinx/AMD e preparação do ambiente de verificação. As seções seguintes também deixam espaço reservado para a inclusão de evidências experimentais (formas de onda e diagramas RTL) e resultados de síntese, a serem coletados posteriormente.

2 Especificação do Sistema

2.1 Resumo do Enunciado

O sistema deve receber duas entradas de dados (**i_x** e **i_y**), cada uma com 8 bits, e produzir em **o_d** o MDC entre esses operandos. O fluxo é controlado por uma entrada de comando **i_go** e uma saída de status **o_rdy**. Enquanto **o_rdy = 1**, o circuito está pronto para receber um novo par de operandos; quando **i_go** é ativado, o circuito captura os dados, executa o algoritmo e devolve **o_rdy = 1** apenas após o resultado ser disponibilizado em **o_d**. A implementação deve ser hierárquica, com módulos separados para topo, controle e caminho de dados, seguindo a convenção de nomes indicada no enunciado (Tabela 1).

2.2 Interface e Convenção de Sinais

A Tabela 1 apresenta um resumo das interfaces externas, enquanto a Tabela 2 registra os principais sinais internos que emergiram do processo de particionamento.

Tabela 1 – Interface externa do sistema **mdc_top**.

Identificador	Tipo	Descrição
i_CLK	Entrada (1 bit)	Clock principal do sistema.
i_RSTn	Entrada (1 bit)	Reset assíncrono ativo em nível baixo.
i_GO	Entrada (1 bit)	Pulso que inicia o processamento.
i_X	Entrada (8 bits)	Operando A sem sinal.
i_Y	Entrada (8 bits)	Operando B sem sinal.
o_D	Saída (8 bits)	Resultado do MDC após a convergência.
o_RDY	Saída (1 bit)	Indica se o módulo está pronto para nova transação.

Tabela 2 – Sinais internos definidos no particionamento controle/caminho de dados.

Sinal	Origem/Destino	Função
w_LD_X	Controle → Datapath	Habilita a carga do registrador r_X .
w_LD_Y	Controle → Datapath	Habilita a carga do registrador r_Y .
w_SEL_X	Controle → Datapath	Seleciona entre entrada externa e resultado de subtração para r_X .
w_SEL_Y	Controle → Datapath	Seleciona entre entrada externa e resultado de subtração para r_Y .
w_X_EQ_Y	Datapath → Controle	Valor lógico indicando igualdade entre r_X e r_Y .
w_X_LT_Y	Datapath → Controle	Valor lógico indicando se r_X < r_Y .
r_X, r_Y	Datapath	Registradores que armazenam os valores intermediários do algoritmo.

3 Metodologia de Projeto RTL

3.1 Algoritmo de Referência em C

O algoritmo de MDC utilizado como ponto de partida segue o método de subtrações sucessivas, incluindo a lógica de handshake com os sinais `i_go` e `o_rdy`. O código completo é apresentado na Listagem 3.1.

Listing 3.1 – Função de referência em C para o cálculo do MDC com handshake

```

1  #include <stdint.h>
2  #include <stdbool.h>
3
4  static bool i_go = false;
5  static uint8_t i_x = 0U;
6  static uint8_t i_y = 0U;
7
8  static uint8_t gcd_subtractive(uint8_t x, uint8_t y)
9  {
10     while (x != y) {
11         if (x < y) {
12             y -= x;
13         } else {
14             x -= y;
15         }
16     }
17     return x;
18 }
19
20 int main(void)
21 {
22     uint8_t o_d = 0U;
23     bool o_rdy = true;
24
25     while (true) {
26         while (!i_go) {
27             /* idle */
28         }
29
30         o_rdy = false;
31         o_d = gcd_subtractive(i_x, i_y);

```



```

32   o_rdy = true;
33   }
34   }

```

3.2 Derivação da Máquina de Estados

A partir do algoritmo, identificaram-se seis estados para o controle: `s_IDLE`, `s_LOAD`, `s_COMPARE`, `s_SUB_X`, `s_SUB_Y` e `s_DONE`. A Figura 1 registra o diagrama de estados desenhado manualmente (imagem fornecida em `media/states.jpg`), enquanto a Tabela 3 detalha as transições e saídas relevantes.

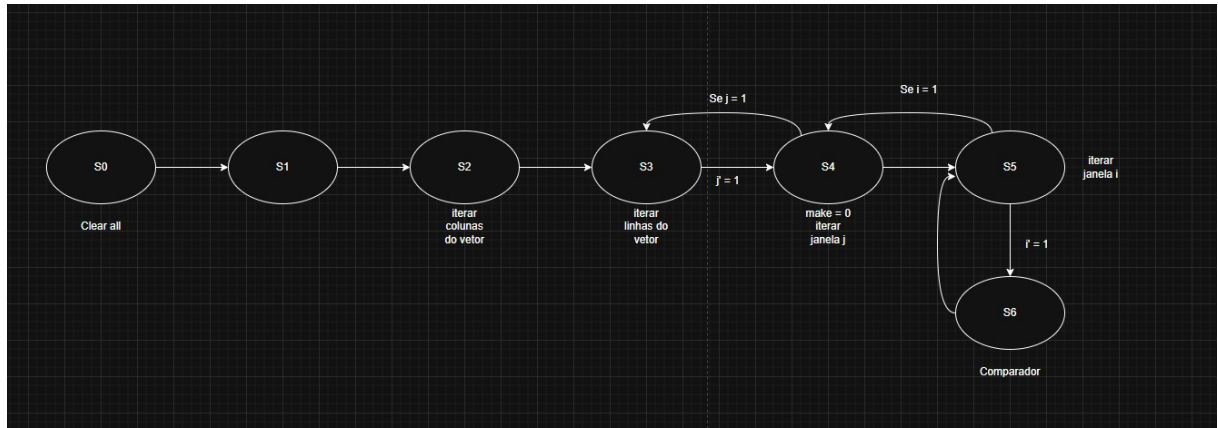


Figura 1 – Diagrama preliminar da máquina de estados de controle.

Tabela 3 – Tabela de transições da FSM de controle.

Estado	Ação principal	Condição	Próximo estado
<code>s_IDLE</code>	<code>o_RDY = 1</code>	<code>i_GO = 1</code> caso contrário	<code>s_LOAD</code> <code>s_IDLE</code>
<code>s_LOAD</code>	Carrega <code>r_X</code> e <code>r_Y</code>	—	<code>s_COMPARE</code>
<code>s_COMPARE</code>	Avalia <code>r_X</code> e <code>r_Y</code>	<code>w_X_EQ_Y = 1</code> <code>w_X_LT_Y = 1</code> demais casos	<code>s_DONE</code> <code>s_SUB_Y</code> <code>s_SUB_X</code> <code>s_COMPARE</code>
<code>s_SUB_X</code>	Atualiza <code>r_X</code> com <code>r_X - r_Y</code>	—	<code>s_COMPARE</code>
<code>s_SUB_Y</code>	Atualiza <code>r_Y</code> com <code>r_Y - r_X</code>	—	<code>s_COMPARE</code>
<code>s_DONE</code>	<code>o_RDY = 1</code> , mantém <code>o_D</code>	<code>i_GO = 0</code>	<code>s_IDLE</code>

3.3 Particionamento do Caminho de Dados

O caminho de dados contém dois registradores (r_X e r_Y), dois blocos de subtração e multiplexadores de entrada. Os subprodutos das subtrações ($w_{SUB_XY} = r_X - r_Y$ e $w_{SUB_YX} = r_Y - r_X$) são selecionados pelo controle para atualizar o registrador apropriado, evitando underflow por meio da lógica de decisão da FSM. As comparações de igualdade ($w_{X_EQ_Y}$) e menor ($w_{X_LT_Y}$) são calculadas continuamente e alimentam o bloco de controle.

4 Arquitetura RTL Proposta

4.1 Visão Geral

A Figura 2 apresenta o diagrama RTL em alto nível, destacando a interação entre o bloco de controle (`mdc_control`) e o caminho de dados (`mdc_datapath`). As setas são etiquetadas com os sinais definidos na Tabela 2.

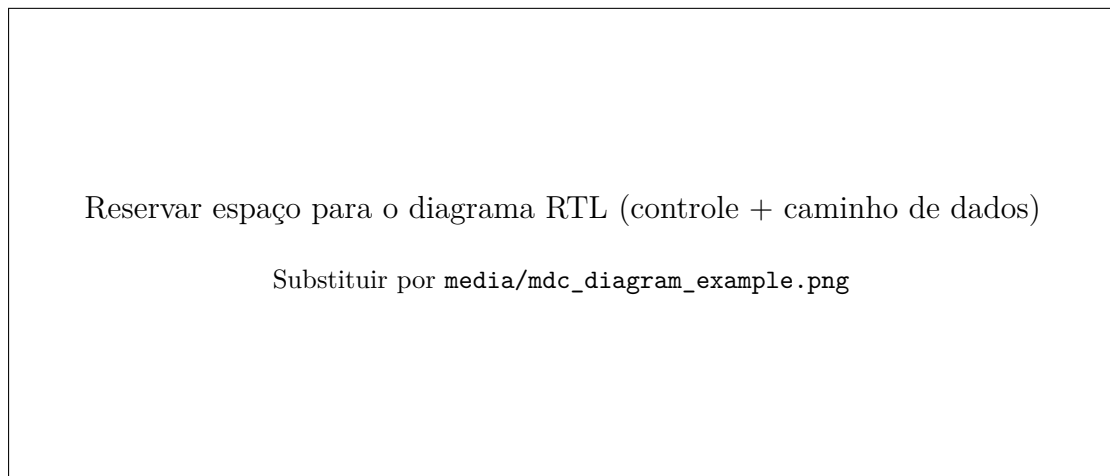


Figura 2 – Particionamento em controle e caminho de dados do sistema MDC.

4.2 Bloco de Controle

O bloco `mdc_control` é responsável por sequenciar o algoritmo, garantindo que apenas operações válidas de subtração sejam realizadas. O uso de estados explícitos facilita a inclusão de novas condições de parada (por exemplo, detecção de operandos nulos) sem alterar o caminho de dados. As saídas `w_LD_X`, `w_LD_Y`, `w_SEL_X` e `w_SEL_Y` são exclusivamente geradas pelo controle e utilizadas como sinais de habilitação e seleção dentro do datapath.

4.3 Caminho de Dados

O caminho de dados mantém os operandos em registradores e atualiza seus valores com base na decisão do controle. As subtrações são realizadas em paralelo, permitindo que o controlador selecione o resultado correto com atraso mínimo. O resultado final é exposto diretamente em `o_result`, que é repassado ao módulo topo.

5 Implementação em VHDL

Os códigos foram escritos seguindo o guia de estilo da avaliação: palavras reservadas em minúsculas, sinais em maiúsculas com prefixos (*i_*, *o_*, *w_*, *r_*), tabulação de dois espaços e comentários em inglês para evitar problemas de codificação.

5.1 Módulo Topo

A Listagem 5.1 mostra o módulo topo `mdc_top`, responsável por conectar os blocos de controle e caminho de dados e disponibilizar a interface externa.

Listing 5.1 – Módulo topo do sistema MDC

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity mdc_top is
5      port (
6          i_clk : in std_logic;
7          i_rstn : in std_logic;
8          i_go  : in std_logic;
9          i_x   : in std_logic_vector(7 downto 0);
10         i_y   : in std_logic_vector(7 downto 0);
11         o_d   : out std_logic_vector(7 downto 0);
12         o_rdy : out std_logic
13     );
14 end entity mdc_top;
15
16 architecture rtl of mdc_top is
17     signal w_LD_X : std_logic;
18     signal w_LD_Y : std_logic;
19     signal w_SEL_X : std_logic;
20     signal w_SEL_Y : std_logic;
21     signal w_X_EQ_Y : std_logic;
22     signal w_X_LT_Y : std_logic;
23     signal w_RESULT : std_logic_vector(7 downto 0);
24 begin
25
26     u_datapath : entity work.mdc_datapath
27         port map (
28             i_clk => i_clk,
```

```

29     i_rstn  => i_rstn,
30     i_x     => i_x,
31     i_y     => i_y,
32     i_ld_x  => w_LD_X,
33     i_ld_y  => w_LD_Y,
34     i_sel_x => w_SEL_X,
35     i_sel_y => w_SEL_Y,
36     o_x_eq_y => w_X_EQ_Y,
37     o_x_lt_y => w_X_LT_Y,
38     o_result => w_RESULT
39 );
40
41 u_control : entity work.mdc_control
42 port map (
43     i_clk  => i_clk,
44     i_rstn => i_rstn,
45     i_go   => i_go,
46     i_x_eq_y => w_X_EQ_Y,
47     i_x_lt_y => w_X_LT_Y,
48     o_ld_x  => w_LD_X,
49     o_ld_y  => w_LD_Y,
50     o_sel_x => w_SEL_X,
51     o_sel_y => w_SEL_Y,
52     o_rdy   => o_rdy
53 );
54
55 o_d <= w_RESULT;
56
57 end architecture rtl;

```

5.2 Máquina de Controle

A Listagem 5.2 apresenta a implementação da FSM derivada na Tabela 3. Os estados são codificados em um tipo enumerado `t_STATE`, e os sinais de controle seguem diretamente a tabela de transições.

Listing 5.2 – Implementação da FSM de controle

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity mdc_control is

```

```

5  port (
6      i_clk    : in  std_logic;
7      i_rstn   : in  std_logic;
8      i_go     : in  std_logic;
9      i_x_eq_y : in  std_logic;
10     i_x_lt_y  : in  std_logic;
11     o_ld_x    : out std_logic;
12     o_ld_y    : out std_logic;
13     o_sel_x   : out std_logic;
14     o_sel_y   : out std_logic;
15     o_rdy     : out std_logic
16 );
17 end entity mdc_control;
18
19 architecture rtl of mdc_control is
20     type t_STATE is (s_IDLE, s_LOAD, s_COMPARE, s_SUB_X, s_SUB_Y, s_DONE);
21     signal r_STATE : t_STATE;
22     signal w_NEXT_STATE : t_STATE;
23
24     signal w_LD_X : std_logic;
25     signal w_LD_Y : std_logic;
26     signal w_SEL_X : std_logic;
27     signal w_SEL_Y : std_logic;
28     signal w_RDY : std_logic;
29 begin
30
31     p_STATE_REG : process(i_clk, i_rstn)
32     begin
33         if i_rstn = '0' then
34             r_STATE <= s_IDLE;
35         elsif rising_edge(i_clk) then
36             r_STATE <= w_NEXT_STATE;
37         end if;
38     end process p_STATE_REG;
39
40     p_STATE_NEXT : process(r_STATE, i_go, i_x_eq_y, i_x_lt_y)
41     begin
42         w_NEXT_STATE <= r_STATE;
43         case r_STATE is
44             when s_IDLE =>
45                 if i_go = '1' then
46                     w_NEXT_STATE <= s_LOAD;

```

```

47     end if;
48     when s_LOAD =>
49         w_NEXT_STATE <= s_COMPARE;
50     when s_COMPARE =>
51         if i_x_eq_y = '1' then
52             w_NEXT_STATE <= s_DONE;
53         elsif i_x_lt_y = '1' then
54             w_NEXT_STATE <= s_SUB_Y;
55         else
56             w_NEXT_STATE <= s_SUB_X;
57         end if;
58     when s_SUB_X =>
59         w_NEXT_STATE <= s_COMPARE;
60     when s_SUB_Y =>
61         w_NEXT_STATE <= s_COMPARE;
62     when s_DONE =>
63         if i_go = '0' then
64             w_NEXT_STATE <= s_IDLE;
65         end if;
66     end case;
67 end process p_STATE_NEXT;
68
69 p_OUTPUTS : process(r_STATE)
70 begin
71     w_LD_X <= '0';
72     w_LD_Y <= '0';
73     w_SEL_X <= '0';
74     w_SEL_Y <= '0';
75     w_RDY <= '0';
76
77     case r_STATE is
78     when s_IDLE =>
79         w_RDY <= '1';
80     when s_LOAD =>
81         w_LD_X <= '1';
82         w_LD_Y <= '1';
83     when s_COMPARE =>
84         null;
85     when s_SUB_X =>
86         w_LD_X <= '1';
87         w_SEL_X <= '1';
88     when s_SUB_Y =>

```

```

89     w_LD_Y <= '1';
90     w_SEL_Y <= '1';
91     when s_DONE =>
92         w_RDY <= '1';
93     end case;
94 end process p_OUTPUTS;
95
96 o_ld_x <= w_LD_X;
97 o_ld_y <= w_LD_Y;
98 o_sel_x <= w_SEL_X;
99 o_sel_y <= w_SEL_Y;
100 o_rdy <= w_RDY;
101
102 end architecture rtl;

```

5.3 Caminho de Dados

A Listagem 5.3 contém a descrição do caminho de dados. Os cálculos são realizados com a biblioteca `numeric_std`, garantindo aritmética sem sinal consistente com a especificação.

Listing 5.3 – Descrição do caminho de dados

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity mdc_datapath is
6      port (
7          i_clk      : in std_logic;
8          i_rstn     : in std_logic;
9          i_x        : in std_logic_vector(7 downto 0);
10         i_y        : in std_logic_vector(7 downto 0);
11         i_ld_x     : in std_logic;
12         i_ld_y     : in std_logic;
13         i_sel_x    : in std_logic;
14         i_sel_y    : in std_logic;
15         o_x_eq_y   : out std_logic;
16         o_x_lt_y   : out std_logic;
17         o_result   : out std_logic_vector(7 downto 0)
18     );
19 end entity mdc_datapath;

```



```

20
21 architecture rtl of mdc_datapath is
22     signal r_X      : std_logic_vector(7 downto 0);
23     signal r_Y      : std_logic_vector(7 downto 0);
24     signal w_NEXT_X : std_logic_vector(7 downto 0);
25     signal w_NEXT_Y : std_logic_vector(7 downto 0);
26     signal w_SUB_XY : std_logic_vector(7 downto 0);
27     signal w_SUB_YX : std_logic_vector(7 downto 0);
28 begin
29
30     w_SUB_XY <= std_logic_vector(unsigned(r_X) - unsigned(r_Y));
31     w_SUB_YX <= std_logic_vector(unsigned(r_Y) - unsigned(r_X));
32
33     with i_sel_x select
34         w_NEXT_X <= i_x    when '0',
35                     w_SUB_XY when others;
36
37     with i_sel_y select
38         w_NEXT_Y <= i_y    when '0',
39                     w_SUB_YX when others;
40
41     p_REGISTERS : process(i_clk, i_rstn)
42     begin
43         if i_rstn = '0' then
44             r_X <= (others => '0');
45             r_Y <= (others => '0');
46         elsif rising_edge(i_clk) then
47             if i_ld_x = '1' then
48                 r_X <= w_NEXT_X;
49             end if;
50             if i_ld_y = '1' then
51                 r_Y <= w_NEXT_Y;
52             end if;
53         end if;
54     end process p_REGISTERS;
55
56     o_x_eq_y <= '1' when r_X = r_Y else '0';
57     o_x_lt_y <= '1' when unsigned(r_X) < unsigned(r_Y) else '0';
58     o_result <= r_X;
59
60 end architecture rtl;

```

5.4 Testbench

O testbench da Listagem 5.4 instancia o módulo topo, gera o clock, aplica vetores de teste e aguarda o término de cada transação antes de iniciar a próxima. As assertivas de verificação podem ser ampliadas conforme necessário.

Listing 5.4 – Testbench funcional para o sistema MDC

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity mdc_tb is
6  end entity mdc_tb;
7
8  architecture sim of mdc_tb is
9      constant c_CLK_PERIOD : time := 10 ns;
10
11     signal w_CLK : std_logic := '0';
12     signal w_RSTn : std_logic := '0';
13     signal w_GO  : std_logic := '0';
14     signal w_X   : std_logic_vector(7 downto 0) := (others => '0');
15     signal w_Y   : std_logic_vector(7 downto 0) := (others => '0');
16     signal w_D   : std_logic_vector(7 downto 0);
17     signal w_RDY : std_logic;
18 begin
19
20     u_dut : entity work.mdc_top
21         port map (
22             i_clk => w_CLK,
23             i_rstn => w_RSTn,
24             i_go  => w_GO,
25             i_x   => w_X,
26             i_y   => w_Y,
27             o_d   => w_D,
28             o_rdy => w_RDY
29         );
30
31     p_CLK : process
32     begin
33         w_CLK <= '0';
34         wait for c_CLK_PERIOD / 2;
35         w_CLK <= '1';
```

```
36     wait for c_CLK_PERIOD / 2;
37 end process p_CLK;
38
39 p_STIMULUS : process
40 begin
41     w_RSTn <= '0';
42     wait for 3 * c_CLK_PERIOD;
43     w_RSTn <= '1';
44     wait for c_CLK_PERIOD;
45
46     w_X <= std_logic_vector(to_unsigned(108, 8));
47     w_Y <= std_logic_vector(to_unsigned(84, 8));
48     wait for c_CLK_PERIOD;
49     w_GO <= '1';
50     wait for c_CLK_PERIOD;
51     w_GO <= '0';
52
53     wait until w_RDY = '1';
54     wait for 2 * c_CLK_PERIOD;
55
56     w_X <= std_logic_vector(to_unsigned(27, 8));
57     w_Y <= std_logic_vector(to_unsigned(18, 8));
58     wait for c_CLK_PERIOD;
59     w_GO <= '1';
60     wait for c_CLK_PERIOD;
61     w_GO <= '0';
62
63     wait for 60 * c_CLK_PERIOD;
64     wait;
65 end process p_STIMULUS;
66
67 end architecture sim;
```

6 Plano de Validação e Síntese

6.1 Validação Funcional

A simulação será conduzida no ModelSim, utilizando o testbench descrito na Seção anterior. Estão planejadas as seguintes etapas:

- Compilação dos arquivos VHDL (`mdc_datapath.vhd`, `mdc_control.vhd`, `mdc_top.vhd`, `mdc_tb.vhd`) com suporte a VHDL-2008.
- Execução do testbench até que `o_RDY` retorne ao nível lógico alto, capturando as formas de onda relevantes (`i_go`, `i_x`, `i_y`, `o_d`, `o_rdy`, `w_X_EQ_Y`, `w_X_LT_Y`, `r_X`, `r_Y`).
- Inclusão das figuras de forma de onda no relatório (Figura 3) com marcação dos pontos de interesse.

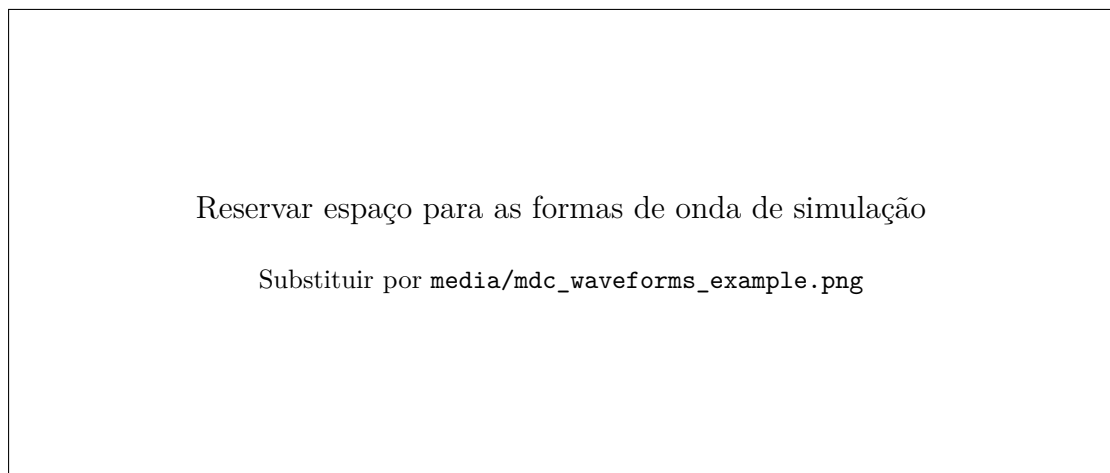


Figura 3 – Espaço reservado para as formas de onda obtidas no ModelSim.

6.2 Síntese e Análise RTL

A síntese será realizada no Quartus Prime, considerando a mesma hierarquia adotada na simulação. Após a compilação, será exportado o diagrama RTL para inclusão na Figura 4, bem como coletadas as métricas de utilização de recursos e frequência máxima (Tabela 4).

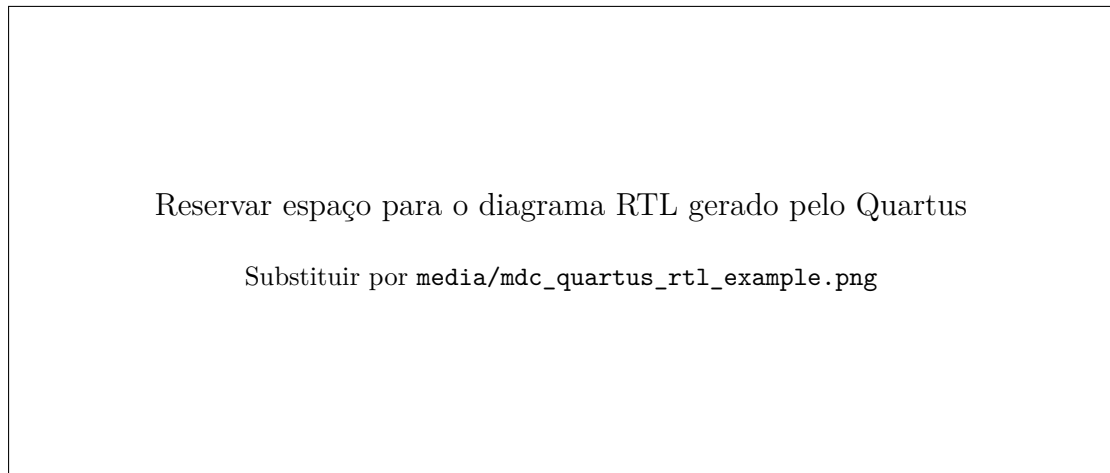


Figura 4 – Espaço reservado para o diagrama RTL gerado pelo Quartus Prime.

Tabela 4 – Template para registrar os custos e o desempenho após a síntese.

Métrica	Valor	Observações
Número de LCs		
Número de LUTs		
Número de FFs		
Frequência máxima (Fmax)		

7 Conclusões Parciais

O projeto encontra-se em estágio funcional, com os blocos de controle e datapath implementados e integrados. A metodologia de projeto RTL foi seguida rigorosamente: partiu-se do algoritmo em C, derivou-se uma FSM de alto nível, identificaram-se os sinais necessários e codificaram-se os módulos em VHDL com a convenção estabelecida. As próximas atividades concentram-se na coleta de evidências experimentais (simulações e síntese) e no refinamento do relatório com base nesses resultados.

Referências

- [1] Xilinx. *VHDL Coding Style Guidelines*. AMD Adaptive Computing Documentation, 2023.
- [2] Intel. *Quartus Prime Standard Edition User Guide*. 2024.
- [3] Siemens EDA. *ModelSim Simulation User Manual*. 2024.